ASSIGNMENT-21.3

2303A52481

SINDHU KATAKAM

BATCH-41

Task 1 – AI-Assisted Pair Programming

PROMPT:

Give code for Expense Tracker

CODE:

```python
# basic_expense_tracker.py
expenses = []
while True:
    print("\n1. Add expense\n2. View all\n3. Exit")
    choice = input("Select: ")

    if choice == "1":
        amount = input("Amount: ")
        category = input("Category: ")
        date = input("Date (YYYY-MM-DD): ")
        expenses.append({"amount": amount, "category": category, "date": date})
        print("Added!")
    elif choice == "2":
        for e in expenses:
            print(e)
    elif choice == "3":
        break
```

OUTPUT:

```
1. Add expense
2. View all
3. Exit
Select: 1
Amount: 23000
Category: food
Date (YYYY-MM-DD): 2026-04-03
Added!

1. Add expense
2. View all
3. Exit
Select: 2
{'amount': '23000', 'category': 'food', 'date': '2026-04-03'}
```

PROMPT:

Write a Python program for a simple Expense Tracker Application that allows a user to record daily expenses along with categories (like food, travel, etc.). First, generate a basic version of the program with simple input and storage. Then review the code and identify its limitations, such as lack of input validation or structure. After that, improve the program by adding features like monthly expense summary, proper input validation, and organizing the code into functions for better readability. Finally, provide both the initial and improved versions and explain how the changes make the program more efficient and user-friendly.

CODE:

```python
# improved_expense_tracker.py
import json
import os
from datetime import datetime
from typing import List, Dict, Optional

DATA_FILE = "expenses.json"
VALID_CATEGORIES = {"food", "travel", "utilities", "entertainment", "health", "other"}

class ExpenseTracker:
    def __init__(self, filename: str = DATA_FILE):
        self.filename = filename
        self.expenses: List[Dict] = self.load_expenses()

    def load_expenses(self) -> List[Dict]:
        """Load expenses from JSON file."""
        if os.path.exists(self.filename):
            try:
                with open(self.filename, "r") as f:
                    return json.load(f)
            except json.JSONDecodeError:
                print("Error reading file. Starting fresh.")
                return []
        return []

    def save_expenses(self):
        """Save expenses to JSON file."""
        with open(self.filename, "w") as f:
            json.dump(self.expenses, f, indent=2)
        print("✓ Data saved.")

    def validate_amount(self, amount_str: str) -> Optional[float]:
        """Validate and convert amount to float."""
```

```python
        try:
            amt = float(amount_str)
            if amt <= 0:
                print("Amount must be positive.")
                return None
            return amt
        except ValueError:
            print("Invalid amount format.")
            return None

    def validate_date(self, date_str: str) -> Optional[str]:
        """Validate date format (YYYY-MM-DD)."""
        try:
            datetime.strptime(date_str, "%Y-%m-%d")
            return date_str
        except ValueError:
            print("Invalid date format. Use YYYY-MM-DD")
            return None

    def validate_category(self, category_str: str) -> Optional[str]:
        """Validate category against allowed list."""
        cat = category_str.lower().strip()
        if cat in VALID_CATEGORIES:
            return cat
        print(f"Invalid category. Allowed: {', '.join(VALID_CATEGORIES)}")
        return None

    def add_expense(self):
        """Add a new expense with validation."""
        print("\n--- Add Expense ---")
```

```python
    while True:
        amount_str = input("Amount (₹): ").strip()
        amount = self.validate_amount(amount_str)
        if amount:
            break

    # Validate category
    while True:
        category = input(f"Category ({', '.join(VALID_CATEGORIES)}): ").strip()
        cat = self.validate_category(category)
        if cat:
            break

    # Validate date
    while True:
        date_str = input("Date (YYYY-MM-DD) [or press Enter for today]: ").strip()
        if not date_str:
            date_str = datetime.now().strftime("%Y-%m-%d")
        date = self.validate_date(date_str)
        if date:
            break

    description = input("Description (optional): ").strip() or "No description"

    expense = {
        "amount": amount,
        "category": cat,
        "date": date,
        "description": description
    }
    self.expenses.append(expense)
```

```python
    # Validate amount
    while True:
        amount_str = input("Amount (₹): ").strip()
        amount = self.validate_amount(amount_str)
        if amount:
            break

    # Validate category
    while True:
        category = input(f"Category ({', '.join(VALID_CATEGORIES)}): ").strip()
        cat = self.validate_category(category)
        if cat:
            break

    # Validate date
    while True:
        date_str = input("Date (YYYY-MM-DD) [or press Enter for today]: ").strip()
        if not date_str:
            date_str = datetime.now().strftime("%Y-%m-%d")
        date = self.validate_date(date_str)
        if date:
            break

    description = input("Description (optional): ").strip() or "No description"

    expense = {
        "amount": amount,
        "category": cat,
        "date": date,
        "description": description
    }
```

```python
    self.expenses.append(expense)
    print("✓ Expense added successfully!")
    self.save_expenses()

def view_all_expenses(self):
    """Display all expenses."""
    if not self.expenses:
        print("No expenses recorded yet.")
        return

    print("\n--- All Expenses ---")
    total = 0
    for i, exp in enumerate(self.expenses, 1):
        print(f"{i}. [{exp['date']}] {exp['category'].upper()}: ₹{exp['amount']} - {exp['descrip
        total += exp['amount']
    print(f"Total: ₹{total:.2f}\n")

def monthly_summary(self):
    """Display expense summary by month."""
    if not self.expenses:
        print("No expenses to summarize.")
        return
    print("\n--- Monthly Summary ---")
    monthly_data: Dict[str, Dict[str, float]] = {}
    for exp in self.expenses:
        month = exp['date'][:7]  # YYYY-MM
        category = exp['category']
        amount = exp['amount']
        if month not in monthly_data:
            monthly_data[month] = {}
        if category not in monthly_data[month]:
```

```python
    # Display organized summary
    for month in sorted(monthly_data.keys()):
        print(f"\n{month}:")
        total = 0
        for cat in sorted(monthly_data[month].keys()):
            amt = monthly_data[month][cat]
            print(f"  {cat.capitalize()}: ₹{amt:.2f}")
            total += amt
        print(f"  Total: ₹{total:.2f}")

def category_summary(self):
    """Display expense summary by category."""
    if not self.expenses:
        print("No expenses to summarize.")
        return

    print("\n--- Category Summary ---")
    category_data: Dict[str, float] = {}

    for exp in self.expenses:
        cat = exp['category']
        if cat not in category_data:
            category_data[cat] = 0
        category_data[cat] += exp['amount']

    total = 0
    for cat in sorted(category_data.keys()):
        amt = category_data[cat]
        print(f"{cat.capitalize()}: ₹{amt:.2f}")
        total += amt
    print(f"Total: ₹{total:.2f}\n")
```

```python
def delete_expense(self):
    """Delete an expense by index."""
    if not self.expenses:
        print("No expenses to delete.")
        return
    self.view_all_expenses()
    try:
        idx = int(input("Enter expense number to delete: ")) - 1
        if 0 <= idx < len(self.expenses):
            deleted = self.expenses.pop(idx)
            print(f"✓ Deleted: ₹{deleted['amount']} ({deleted['category']})")
            self.save_expenses()
        else:
            print("Invalid index.")
    except ValueError:
        print("Invalid input.")
def run(self):
    """Main application loop."""
    while True:
        print("\n=== Expense Tracker ===")
        print("1. Add expense")
        print("2. View all expenses")
        print("3. Monthly summary")
        print("4. Category summary")
        print("5. Delete expense")
        print("6. Exit")
        print("=" * 25)
        choice = input("Select option (1-6): ").strip()
        if choice == "1":
            self.add_expense()
        elif choice == "2":
            self.view_all_expenses()
```

```python
        elif choice == "2":
            self.view_all_expenses()
        elif choice == "3":
            self.monthly_summary()
        elif choice == "4":
            self.category_summary()
        elif choice == "5":
            self.delete_expense()
        elif choice == "6":
            print("Thank you for using Expense Tracker!")
            break
        else:
            print("Invalid choice. Try again.")

f __name__ == "__main__":
    tracker = ExpenseTracker()
    tracker.run()
```

OUTPUT:

```
PS C:\Users\mohan\OneDrive\Documents\Desktop\ai codding> & C:/Users/mohar
ocuments/Desktop/ai codding/ass21.3.py"

=== Expense Tracker ===
1. Add expense
2. View all expenses
3. Monthly summary
4. Category summary
5. Delete expense
6. Exit
==========================
Select option (1-6): 1

--- Add Expense ---
Amount (₹): 20000
Category (travel, other, entertainment, utilities, health, food): travel
Date (YYYY-MM-DD) [or press Enter for today]: 2025-06-23
Description (optional): --NA--
✓ Expense added successfully!
✓ Data saved.
```

JUSTIFICATION

An expense tracker teaches you how to build real-world software. A basic version loses data and accepts wrong input. By adding validation, saving data, organizing code, and creating summaries, you learn skills used in professional apps: preventing errors, keeping data safe, writing clear code, and analyzing information to find patterns.

Task 2 – Version Control Integration

```
sindhu@sindhu-750XGK:~/Downloads/assistantcoding-2020631770846292-3-001$ cd /home/sindhu/Downloads/grade-calculator
 &6. git commit -m "Initial grade calculator with average"
 Switched to branch 'main'.

sindhu@sindhu-750XGK:~/Downloads/assistantcoding$ cd /home/sindhu/Downloads/sindhu && git checkout -b feature-ai-grade &&
sindhu@sindhu-750XGK:~/Downloads/grade-calculator$ /cd /home/sindhu/Downloads/ssindhu &&
sindhu@sindhu-750XGK:~/Downloads/grade-calculator $&
sindhu@sindhu-750XGK:~/Downloads/assistantcodings cd /home/sindhu/Downloads/sindhu && git checkout -b feature-ai-grade &&
sindhu@sindhu-750XGK:~/Downloads/grade-calculator &

sindhu@sindhu-750XGK:~/Downloads/assistantcoding$ cd /home/sindhu/Downloads/sindhu && git checkout -b feature-ai-grade &&
Switched to a new branch 'feature-ai-grade'
sindhu@sindhu-750XGK:~/Downloads/grade-calculator$ /cd /home/sindhu/Downloads/grade-calculator &&
sindhu@sindhu-750XGK:~/Downloads/grade-calculator
marks = list(map(int, input("Enter marks: ").split()))
total = sum(marks)
avg = total / (len(marks))
grade = ""
if avg >= 90:
elif avg = 80:
elif avg = 70:
elif avg = 60:
elif avg = 70:
elif avg = 60:
elif avg = 'E'
else:
  grade = 'F'
print("Average:`,avg)
print("Percentage", percentage)
print("Grade", grade)
sindhu@sindhu-750XGK:~/Downloads/grade-calculator$
```

PROMPT:

Add grade classification and percentage calculation to the program.

CODE:

```
marks = list(map(int, input("Enter marks: ").split()))

total = sum(marks)
avg = total / len(marks)
percentage = (total / (len(marks)*100)) * 100

if avg >= 90:
    grade = 'A'
elif avg >= 75:
    grade = 'B'
elif avg >= 50:
    grade = 'C'
else:
    grade = 'F'

print("Average:", avg)
print("Percentage:", percentage)
print(“Grade:”, grade)
```

OUTPUT:

```
sindhu@sindhu-750XGK:~/Downloads/quiz-management$ git checkout -b feature-score
Switched to a new branch 'feature-score'

sindhu@sindhu-750XGK:~/Downloads/quiz-management$ git add .
Switched to a new branch 'feature-score'

sindhu@sindhu-750XGK:~/Downloads/quiz-management$ git commit -m "Addded score calculation"
[feature-score 8ab91cd] Added score calculation
 1 file changed, 12 insertions(+), 1 deletion(-)

sindhu@sindhu-750XGK:~/Downloads/quiz-management$ git checkout main
Switched to branch 'main'
8ab91cd | -1#*********-
 └ 1 file changed, 11 inserions-1,deletion(-)

sindhu@sindhu-750XGK:~/Downloads/quiz-management$ git merge feature-score
Capital of India? Delhi
2+2 4
Correct
Final Score: 1

sindhu@sindhu-750XGK:~/Downloads/quiz-management$ git log --oneline --decorate --graph --all --max-count=5
• 8ab91cd (HEAD -> main, feature-score)
└ > Added score calculation *****$
+ 2f4a7d1 Initial quiz management system

•sindhu@sindhu-750XGK:~/Downloads$
```

JUSTIFICATION:

Branching safe feature development, Merge integration workflow, AI faster feature addition.

Task 3 – AI for Commit Message Generation

```python
import re

password = input("Enter password: ")

if len(password) < 8:
    print("Weak password")
elif not re.search(r'[!@#$%^&*]', password):
    print("Add special character")
else:
    print("Strong password")
```

Prompt:

Generate a commit message for adding password validation with length and special character rules

AI Committed Message:

```
-750XGK:~/Downloads/aiasistantcoding-20260312T084629Z-3-001$
av-750XGK:~/Downloads/aiassistantcoding-20260312T084629Z-3-001$ git comm
t -m "feat: add password validation with minimum length and special charac
```

Human Version:

Added strong password validation rules

Justification:

AI more structured (uses “feat:” convention), Human → less descriptive

Task 4 – Pair Programming Simulation

```
questions = {
    "Capital of India?": "Delhi",
    "2+2?": "4"
}

for q in questions:
    ans = input(q + " ")
    if ans == questions[q]:
        print("Correct")
    else:
    print("Wrong")
```

AI Prompt (Student B):

Add score calculation and input validation to the quiz program.

Final Code:

```python
questions = {
    "Capital of India?": "Delhi",
    "2+2?": "4"
}

score = 0

for q in questions:
    ans = input(q + " ").strip()

    if ans.lower() == questions[q].lower():
        print("Correct")
        score += 1
    else:
        print("Wrong")

print("Final Score:", score)
```

Git WorkFlow:

```
sindhu@sindhu-750XGK:~/Downloads/quiz-management$ git checkout -b feature-score
Switched to a new branch 'feature-score'

sindhu@sindhu-750XGK:~/Downloads/quiz-management$ git add .

sindhu@sindhu-750XGK:~/Downloads/quiz-management$ git commit -m "Added score calculation"
[feature-score 8ab91cd] Added score calculation
 1 file changed, 12 insertions(+), 1 deletion(-)

sindhu@sindhu-750XGK:~/Downloads/quiz-management$ git checkout main
Switched to branch 'main'

sindhu@sindhu-750XGK:~/Downloads/quiz-management$ git merge feature-score
Updating 2f47ad1..8ab91cd
Fast-forward
 quiz.py | 12 ***********
 1 file changed, 11 insertions(+), 1 deletion(-)

quiz.py |2
Wrong
Final Score: 1

sindhu@sindhu-750XGK:~/Downloads/quiz-management$ git log --oneline --decorate --graph --all --max-count=5
* 8ab91cd (HEAD -> main, feature-score) Added score calcu
└─ > 2f47ad1 Initial quiz management system

sindhu@sindhu-750XGK:~/Downloads/quiz-management$
```

OUTPUT:

```
sindhu@sindhu-750XGK:~/Downloads$ /usr/bin/python3 /home/sindhu/Downloads/assistantcoding-
2020631770846292-3-001/assistantcoding65.py
Capital of India? Y
Wrong
2+2? 4
Correct
Final Score: 1

sindhu@sindhu-750XGK:~/Downloads$
```

JUSTIFICATION:

This task demonstrates measurable output through score calculation, ensuring that results can be clearly evaluated and tracked. It also incorporates validation to improve user experience by handling inputs effectively and reducing errors. Additionally, it reflects real-world collaboration by using Git workflows such as branching, committing, and merging, which are essential practices in team-based software development.

Task 5: AI-Assisted API Documentation and Readability

Prompt:

Add docstrings and comments to Flask API endpoints and optionally integrate Swagger documentation.

```python
from flask import Flask, jsonify, request

app = Flask(__name__)

# In-memory storage for service requests
requests = []
request_id_counter = 1

@app.route('/status')
def status():
    """
    GET /status
    Returns the API status.

    Responses:
        200: {'status': 'ok', 'message': 'API is running'}
    """
    return jsonify({'status': 'ok', 'message': 'API is running'})

@app.route('/requests', methods=['POST'])
def create_request():
    """
    POST /requests
    Creates a new service request.

    Request Body (JSON):
        {'title': 'string', 'priority': 'string'}

    Responses:
        201: New request object with 'id', 'title', 'priority'
        400: {'error': 'Missing title or priority'}
    """
    global request_id_counter
    data = request.get_json()
    # Validate input data
    if not data or 'title' not in data or 'priority' not in data:
        return jsonify({'error': 'Missing title or priority'}), 400

    # Create a new request object
    new_request = {
        'id': request_id_counter,
        'title': data['title'],
        'priority': data['priority']
    }
    requests.append(new_request)
    request_id_counter += 1
    return jsonify(new_request), 201

@app.route('/requests', methods=['GET'])
def get_requests():
    """
    GET /requests
    Retrieves all service requests.

    Responses:
        200: A list of all request objects.
    """
    return jsonify(requests)

@app.route('/requests/<int:request_id>', methods=['PUT'])
def update_request(request_id):
    """
    PUT /requests/<request_id>
    Updates an existing service request by its ID.

    Path Parameters:
        request_id (int): The ID of the request to update.
```

```python
    # Validate input data
    if not data or 'title' not in data or 'priority' not in data:
        return jsonify({'error': 'Missing title or priority in request body'}), 400

    # Find and update the request
    for req in requests:
        if req['id'] == request_id:
            req['title'] = data['title']
            req['priority'] = data['priority']
            return jsonify(req), 200
    return jsonify({'error': 'Request not found'}), 404

@app.route('/requests/<int:request_id>', methods=['DELETE'])
def delete_request(request_id):
    """
    DELETE /requests/<request_id>
    Deletes a service request by its ID.

    Path Parameters:
        request_id (int): The ID of the request to delete.

    Responses:
        200: {'message': 'Request with id {request_id} deleted successfully'}
        404: {'error': 'Request not found'}
    """
    global requests
    original_length = len(requests)
    # Filter out the request to be deleted
    requests = [req for req in requests if req['id'] != request_id]
    if len(requests) < original_length:
        return jsonify({'message': f'Request with id {request_id} deleted successfully'}), 200
    return jsonify({'error': 'Request not found'}), 404

if __name__ == '__main__':
    # In Colab, you might need to use a different host to make it accessible
    # from the Colab environment. '0.0.0.0' is generally used for external access.
    # However, for local testing within Colab, '127.0.0.1' or even not specifying
    # the host might work depending on the Colab environment setup.
    # For a simple run, you can just use app.run(debug=True).
    # If you intend to expose it via ngrok or similar, you'd configure that separately.
    print("To run the app, execute 'python your_file_name.py' in a terminal if this were a script.")
    print("In Colab, you can run it directly, but accessing it from outside Colab cells might require port forwarding or ngrok.")
    app.run(host='0.0.0.0', port=5000, debug=True)
```

Output:

```
To run the app, execute 'python your_file_name.py' in a terminal if this were a script.
In Colab, you can run it directly, but accessing it from outside Colab cells might require port forwarding or ngrok.
 * Serving Flask app '__main__'
 * Debug mode: on
INFO:werkzeug:WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://172.28.0.12:5000
INFO:werkzeug:Press CTRL+C to quit
INFO:werkzeug: * Restarting with watchdog (inotify)
```

Explanation:

Docstrings improve code readability by clearly explaining the purpose, inputs, and outputs of each function, making it easier for developers to understand and maintain the code. Swagger provides automatic API documentation, allowing developers to quickly explore endpoints, understand request and response formats, and integrate with the API more efficiently.